



Triple Jump (Solution)

For any i and j , we consider when they are possibly used as $a = i$ and $b = j$ (in any one of the queries).

First, if there exists k satisfying $i < k < j$ and $A_k \geq A_j$, we do not need to consider the solution with $a = i$ and $b = j$ because if we put $b = k$, we get a solution which is not worse than it.

Second, if $A_i \leq A_j$ and there exists k satisfying $i < k < j$ and $A_i \leq A_k \leq A_j$, we do not need to consider the solution with $a = i$ and $b = j$ because if we put $a = k$, we get a solution which is not worse than it.

By the above two reasons, the number of pairs i, j we need only consider is $O(N)$. We can list all such pairs if for each i from N to 1, we keep the set of candidate j 's. We get a list of pairs i, j which are candidates of a, b .

Then, we process the queries. For each t from N to 1, we keep a sequence v . Here the x -th value of the sequence v is defined as the maximum of the sum $A_a + A_b + A_c$ for a, b, c ($b - a \leq c - b$) satisfying $t \leq a < b < c = x$.

Once we can keep the sequence v , it is easy to answer the queries.

Let us consider how the sequence v varies if we change t from $s + 1$ to s . For each pair i, j of a candidate of a, b as above satisfying $s = i$, for every k with $2j - i \leq k$, we have

$$v_k = \max(v_k, A_i + A_j + A_k)$$

Using Segment Tree, we can update a candidate in $O(\log N)$ time. We can calculate the answer for each query in $O(\log N)$ time. Therefore, we can solve this task in $O((N + Q) \log N)$ time.



Remittance (Solution)

Subtask 1

We can solve Subtask 1 if we search all possible ways of remittance. In fact, if $A_i \leq 5$, it can be proved that the amount of money in every house does not exceed 9 yen if we use the remittance service in any way. Since $N \leq 7$, there are at most 10^N possible states of the amount of money in each house. We can search all of them in time.

Subtasks 2 and 3

In order to solve Subtask 2, we shall solve equations. Let us consider the process where the amount of money in a house becomes B yen. Let X_i be the amount of money sent by House i . House i pays $2X_i$ yen, and receives X_{i-1} yen. Here we put $X_0 = X_N$. Then we have the equation

$$B_i - A_i = X_{i-1} - 2X_i.$$

Let us solve it.

Taking the sum of the equation times 2^{i-1} for every House i , we get

$$\sum_{i=1}^N 2^{i-1} (B_i - A_i) = (1 - 2^N) X_N.$$

From this, we get $X_N = X_0$. Then, solving the equation for each House i in order, we get X_i .

Each X_i must be a non-negative integer. If this condition is not satisfied, the answer is No. In fact, if every X_i is a non-negative integer and at least one of B_i is non-zero, the answer is Yes. We shall prove this.

We consider the operations in the opposite direction. We shall transform B into A .

If some entry of the sequence B is non-zero and the sequence A is different from B , we have $B_{i+1} > 0$ and $X_i > 0$ for some i . In fact, if this condition is not satisfied, we have $B_{i+1} = 0$ for some i with $X_i > 0$. Since $A_{i+1} \geq 0$, we have $X_{i+1} > 0$ by the above equation. Hence we have $X_i > 0$ for every i , and $B_i = 0$, which is absurd.

For such i , we apply the operation which is opposite to sending 1 yen from House i to House $i + 1$ to the sequence B . Then B_{i+1} decreases by 1, B_i increases by 2, and X_i decreases by 1. Repeating this operation, we can transform B into A .

Therefore, if all entries of B are 0, the answer is Yes if all entries of A are 0. Otherwise, the answer is No.



In Subtask 2, the value of

$$\sum_{i=1}^N 2^{i-1} (B_i - A_i)$$

can be calculated as a 64 bit integer. In Subtask 3, this value can be very large. Even in such cases, we can solve the task in $O(N)$ time using multiple precision integers.

The sample source file `2019-open-remittance-sol1.cpp` implements this solution.

Another solution

In fact, we can solve this task by the following simulations.

For each House i , if it has a yen, send $\left\lceil \frac{a - B_i}{2} \right\rceil$ yen if $a > B_i$. Repeat this for each i . We increase i by 1 in each step. When i becomes N , we return to $i = 1$. Note that the amount of money in the House can be -1 in this process.

If the answer is Yes, after repeating this process in $N + O\left(\log \sum_{i=1}^N A_i\right)$ steps, the amount of money in every House becomes B . Let us prove this fact. During this simulation, the amount of money sent from House i does not exceed X_i yen. Thus, from the intermediate status, there is a way to make the amount of money in every House be B . After one cycle of i , the amount of money in every House does not exceed B except possibly for one House. Moreover, the excess for that House becomes half after each step. Therefore, after $N + O\left(\log \sum_{i=1}^N A_i\right)$ steps, the amount of money in every House becomes B .

By the above argument, we can also show that the answer is Yes if we can make the amount of money in every House be B and the sequence B contains a non-zero value.

The sample source file `2019-open-remittance-sol2.cpp` implements this solution.



Virus Experiment (Solution)

Solution

Preprocessing the direction of the wind, for each inhabitant, if we know the status (infected or uninfected) of 4 adjacent inhabitants, we can determine whether he or she gets infected or not in $O(1)$ time.

We consider a directed graph whose vertices are inhabitants and edges are the relations “if only this inhabitant is infected, that inhabitant is finally infected.” We decompose it into strong connected components. We need to calculate the minimum number of vertices in a strong connected component of outdegree 0. We also need to calculate the number of strong connected components attaining the minimum number of vertices.

This can be done by the following DFS (depth-first search).

1. We construct a union-find, where we can list all vertices in a union by a merging technique.
2. For a vertex, if every vertex in a union where this vertex belong to is infected, we search the next vertex to be infected.
3. If we arrive at a vertex where we already visited, we stop searching if it is not an ancestor of DFS. If it is an ancestor of DFS, we merge the union of all vertices in the path we followed. (We can determine whether it is an ancestor or not in $O(1)$ time.)

Then, a union means “the set of vertices which infect each other.”

There is a difficulty in speeding up the step 2, especially in listing the vertices adjacent to a union. There are two cases:

- (A) The union consists of one vertex (i.e., we arrive at a vertex for the first time).
- (B) We are considering union which was created by merging two already-existing unions.

The case (A) can be done in $O(1)$ time.

Concerning the case (B), if a vertex is adjacent to only one of two unions to be merged, it was already searched by DFS before merging the unions. Therefore, we need only consider a vertex which is adjacent to both of the two unions. We calculate all vertices which are adjacent to the smaller union. By merging technique, the amortized time complexity is $O(HW \log(HW))$.



Solution (randomized algorithm)

Take a sequence of vertices A where every vertex appears exactly once.

For each of the vertices $A_0, A_1, \dots$, we do a naive simulation under the condition that it is the only initial infected vertex. In a simulation where A_k is the initial infected vertex, if the vertex A_l is infected and $l < k$, then we quit the simulation.

For simulations which are not quit on the way, we need to calculate the minimum number of vertices which are finally infected, and the number of simulations which attain the minimum.

In this solution, the worst time complexity is $O((HW)^2)$. If we take the initial sequence A randomly, the time complexity becomes roughly $O(HW \log(HW))$.

The sample source file `2019-open-virus-sol1.cpp` implements this solution.

Another solution

For another solution, see the sample source file `2019-open-virus-sol2.cpp`.